

SENTINEL ACADEMIA

Plataforma Universitaria de Quejas con IA

Conexiones de Arquitectura

Guía paso a paso

Resumen: Este documento describe cómo conectar todos los servicios de la arquitectura de Sentinel AcademIA: desde el navegador del usuario hasta los servicios de AWS (Lambda, S3, Cognito, SNS) y Oracle Cloud Infrastructure (Generative AI, Document AI).

Backend: **Python 3.12** en AWS Lambda
Frontend: **Vue 3 + TypeScript** en S3
LLM: **Cohere Command R** vía OCI

20 de junio de 2026

Índice

1. Resumen de la Arquitectura	2
2. Diagrama Completo de Arquitectura	3
3. Conexiones Paso a Paso (los 10 cables)	3
3.1. Conexión 1: Usuario → Frontend	4
3.2. Conexión 2: Usuario → API Gateway (10 Lambdas)	4
3.3. Conexión 3: Lambda → Cognito (Auth)	5
3.4. Conexión 4: Lambda → DynamoDB (NoSQL)	5
3.5. Conexión 5: Lambda → S3 (Evidencia + RAG)	5
3.6. Conexión 6 (async): create_queja → SQS → analyze_queja	6
3.7. Conexión 7: analyze_queja → OCI Cohere (LLM real)	6
3.8. Conexión 8: analyze_queja → SNS → Email	6
3.9. Conexión 9 (async): S3 Event → file_processor	7
3.10. Conexión 10: notify_email → S3 (HTML preview)	7
4. Configuración AWS Requerida	7
5. Orden de Despliegue Recomendado	7
6. Checklist de Verificación	8

1. Resumen de la Arquitectura

Sentinel AcademIA es una plataforma **serverless multi-tenant** para gestión de quejas universitarias con análisis automático de IA. La arquitectura se divide en **seis zonas**:

1. **Cliente (ClientEdge)**: navegador del usuario, frontend Vue 3
2. **AWS Lambda** (10 funciones): lógica de negocio serverless
3. **AWS Cognito**: autenticación de usuarios
4. **AWS S3 + DynamoDB + SQS**: almacenamiento y colas
5. **AWS SNS + Email**: notificaciones a las autoridades
6. **Oracle OCI**: LLM (Cohere) + Document AI (OCR)

2. Diagrama Completo de Arquitectura

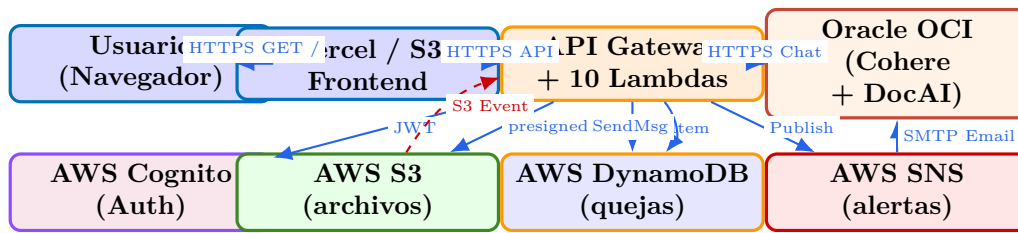


Figura 1: Diagrama completo de la arquitectura con todas las conexiones entre servicios.

3. Conexiones Paso a Paso (los 10 cables)

3.1. Conexión 1: Usuario → Frontend

Desde → Hasta

Desde: Navegador del usuario (Chrome, Firefox, Safari)
Hacia: Frontend Vue 3 desplegado en S3 (Vercel opcional)
Protocolo: HTTPS (puerto 443)



Qué datos fluyen:

- **Request:** el navegador pide `https://sentinel-app.com`
- **Response:** S3/Vercel devuelve HTML, CSS, JS del frontend

Configuración:

1. Crear bucket S3: `sentinel-frontend-dev-{account-id}`
2. Habilitar **Static Website Hosting**
3. Configurar **Bucket Policy** para lectura pública
4. Subir: `aws s3 sync dist/ s3://sentinel-frontend-dev-{id}/`

3.2. Conexión 2: Usuario → API Gateway (10 Lambdas)

Desde → Hasta

Desde: Navegador
Hacia: API Gateway → 10 AWS Lambdas
Protocolo: HTTPS REST



Las 10 Lambdas (cada una = 1 endpoint):

1. `health_check` — GET `/health`
2. `create_queja` — POST `/api/quejas`
3. `get_queja` — GET `/api/quejas/{id}`
4. `list_quejas` — GET `/api/quejas`
5. `analyze_queja` — Consumer de SQS
6. `upload_url` — POST `/api/quejas/{id}/upload-url`
7. `file_processor` — Consumer de S3
8. `chat` — POST `/api/chat`
9. `dashboard_metrics` — GET `/api/dashboard/metrics`
10. `notify_email` — Consumer de SNS

3.3. Conexión 3: Lambda → Cognito (Auth)

Desde → Hasta

Desde: Cualquier Lambda
Hacia: AWS Cognito User Pool
Protocolo: HTTPS (boto3 SDK)



Qué datos fluyen:

- **Request:** `cognito-idp.get_user(AccessToken=...)`
- **Response:** atributos del usuario (sub, email, custom:tenant)

Estado actual: documentado pero el demo usa `X-Tenant-ID` en headers (modo single-tenant simplificado). Para producción, configurar User Pool con atributo personalizado `custom:tenantId` y pre-token-generation Lambda.

3.4. Conexión 4: Lambda → DynamoDB (NoSQL)

Desde → Hasta

Desde: Cualquier Lambda que lea/escriba quejas
Hacia: DynamoDB Table `sentinel-quejas-dev`
Protocolo: HTTPS (boto3 SDK)



Esquema single-table (PK/SK):

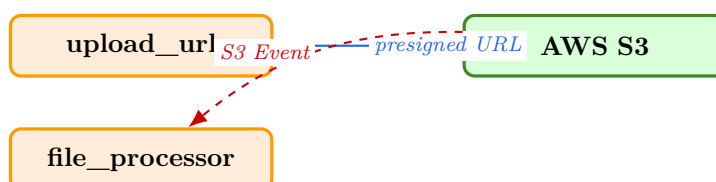
```

PK: TENANT#{tenantId}#QUEJA#{quejaId}
SK: QUEJA#{quejaId}
GSI1PK: TENANT#{tenantId}#STATUS#{status}
GSI1SK: createdAt
Atributos: titulo, descripcion, analysis, ...
  
```

3.5. Conexión 5: Lambda → S3 (Evidencia + RAG)

Desde → Hasta

Desde: Lambda `upload_url`, `file_processor`
Hacia: S3 `sentinel-files-dev-{account-id}`
Protocolo: HTTPS + presigned URLs



Prefijos en S3:

- `tenants/{tenantId}/quejas/{quejaId}/...` (evidencia)
- `tenants/{tenantId}/config.json` (config del tenant)
- `tenant-knowledge/{tenantId}/{docId}.pdf` (PDFs del RAG)
- `emails/{id}.html` (previews HTML)

3.6. Conexión 6 (async): `create_queja` → SQS → `analyze_queja`

Desde → Hasta (ASYNC)

Desde: Lambda `create_queja`

Hacia: SQS Queue → Lambda `analyze_queja`

Protocolo: AWS SQS (asíncrono)



Patín asíncrono:

- `create_queja` responde 202 al usuario inmediatamente
- Encola el ID de la queja en SQS
- `analyze_queja` procesa en background
- Si falla → DLQ (max 3 reintentos, 14 días)

3.7. Conexión 7: `analyze_queja` → OCI Cohere (LLM real)

Desde → Hasta

Desde: Lambda `analyze_queja`

Hacia: OCI Generative AI (Cohere Command R)

Protocolo: HTTPS con firma RSA-SHA256



Endpoint: `https://inference.generativeai.us-chicago-1.oci.oraclecloud.com/20231130/actions/`

Implementación: `services/oci_http_client.py` (sin SDK de 229MB) usa solo cryptography + urllib.

3.8. Conexión 8: `analyze_queja` → SNS → Email

Desde → Hasta (NOTIFICACIÓN)

Desde: Lambda `analyze_queja`

Hacia: SNS Topic → Email (aracely.ayala@utec.edu.pe)

Protocolo: SNS Publish + Email Subscription



Topic: `arn:aws:sns:us-east-1:227165337884:sentinel-alertas-dev`

Suscripción: `aracely.ayala@utec.edu.pe` (email)

3.9. Conexión 9 (async): S3 Event → file_processor

Desde → Hasta (ASYNC)

Desde: S3 (ObjectCreated)
Hacia: Lambda file_processor
Protocolo: S3 Event Notification



Configuración: aws s3api put-bucket-notification-configuration (SAM no permite S3 → Lambda directo por circular dep)

3.10. Conexión 10: notify_email → S3 (HTML preview)

Desde → Hasta

Desde: Lambda notify_email (consumer SNS)
Hacia: S3 sentinel-emails-dev-{id}/emails/
Protocolo: S3 PutObject



Output: URL pública <https://...s3.amazonaws.com/emails/{id}.html>

4. Configuración AWS Requerida

Resumen de servicios AWS

Servicio	Nombre	Región
Lambda (10)	sentinel-*dev	us-east-1
DynamoDB	sentinel-quejas-dev	us-east-1
DynamoDB	sentinel-knowledge-chunks-dev	us-east-1
S3	sentinel-files-dev-{id}	us-east-1
S3	sentinel-knowledge-dev-{id}	us-east-1
S3	sentinel-frontend-dev-{id}	us-east-1
S3	sentinel-emails-dev-{id}	us-east-1
SQS	sentinel-analysis-dev	us-east-1
SQS	sentinel-analysis-dlq-dev	us-east-1
SNS	sentinel-alertas-dev	us-east-1
API Gateway	sentinel-api-dev	us-east-1
Cognito (futuro)	sentinel-academia-users	us-east-1

5. Orden de Despliegue Recomendado

1. **Frontend** (cd web && npx vite build && aws s3 sync)
2. **DynamoDB Tables** (single-table + knowledge chunks)

3. **S3 Buckets** (files, knowledge, frontend, emails)
4. **SQS Queues** (análisis + DLQ)
5. **SNS Topic + suscripción** (alertas + aracely)
6. **Lambdas** (deploy con SAM)
7. **API Gateway** (rutas)
8. **Permisos Lambda** (S3, SNS, DynamoDB, SQS, OCI signing)
9. **Variables de entorno** (OCI_USER_OCID, etc.)
10. **Private key de OCI** en /var/task/oci_key.pem
11. **Test E2E** (python3 scripts/test_e2e_flow.py)

6. Checklist de Verificación

- ✓ Frontend Vue 3 desplegado y accesible
- ✓ Las 10 Lambdas creadas y respondiendo
- ✓ DynamoDB con las 2 tablas creadas
- ✓ S3 con los 4 buckets y políticas correctas
- ✓ SQS con la cola principal y DLQ
- ✓ SNS con suscripción de aracely confirmada
- ✓ OCI Cohere responde (test con la private key)
- ✓ PDFs del reglamento ingestados (68 chunks en DDB)
- ✓ aracely.ayala@utec.edu.pe recibe emails con link al HTML
- ✓ Frontend detecta tenant automáticamente por dominio
- ✓ ACADEMICA requiere cursoCodigo
- ✓ INFRAESTRUCTURA espera 10 similares antes de notificar
- ✓ LLM real genera recomendaciones en cada email
- ✓ HTML preview público en S3